Module 7 — HIJING Glauber & Analysis

Supplementary to the sPHENIX Standalone Course

1. Moving from ASCII to ROOT

Use the ASCII dump only for debugging. For analysis, write a ROOT TTree directly — it's orders of magnitude faster to read and indexable.

```
// In a C++ wrapper that calls HIJING through Fortran hooks
extern "C" { void hijset_(float*,char*,char*,char*,int*,int*,int*,int*,int,int,int);
               void hijing_(char*,float*,float*,int); }
extern "C" struct { int natt; } himain1_;
extern "C" struct { int katt[4][130000]; float patt[4][130000]; } himain2_;

TFile f("hijing.root","RECREATE");
TTree t("evt","HIJING");
int    natt, katt[100000][4];
float  patt[100000][4];
float  b;
t.Branch("b", &b);
t.Branch("natt", &natt);
t.Branch("katt", katt, "katt[natt][4]/I");
t.Branch("patt", patt, "patt[natt][4]/F");
```

2. Glauber scaling

In Heavy-Ion collisions the number of participant nucleons N_part and binary collisions N_coll are the two key geometric quantities:

e	Scales with	Why
e yield	N_part / 2	Each wounded nucleon contributes independently
ss yield	N_coll	Each NN collision can produce a hard process
rapidity	mixture	Mostly soft + some hard

yield	N_coll × R_AA	R_AA<1 signals quenching

3. Event plane and flow

Azimuthal anisotropy in heavy-ion collisions comes from the almond-shaped overlap region. It is quantified by Fourier coefficients v_n of $dN/d\phi$:

```
dN/dϕ = N_0 · (1 + 2·Σ v_n cos[n(ϕ - Ψ_n)])

v_n = ■cos[n(ϕ - Ψ_n)]■
```

Elliptic flow (n=2) is the largest and most studied. The event plane Ψ_2 is reconstructed from the Q-vector:

```
Q_{2x} = Σ_i cos(2 ϕ_i)
Q_{2y} = Σ_i sin(2 ϕ_i)
Ψ_2   = (1/2) · atan2(Q_{2y}, Q_{2x})
```

4. v_2 vs p_T with TProfile

```
TProfile v2("v2", "v_{2} vs p_{T}", 30, 0, 6);
while (next_event()) {
    double Qx=0, Qy=0;
    for (auto& p : particles) { Qx += cos(2*p.phi); Qy += sin(2*p.phi); }
    double psi2 = 0.5 * atan2(Qy, Qx);
    for (auto& p : particles) v2.Fill(p.pt, cos(2*(p.phi - psi2)));
}
```

Note: TProfile stores running means per bin: Fill(x, y) records a y value at x, and GetBinContent returns the average y in that bin. It also handles error propagation (RMS / $\sqrt{N}$) for you.

5. Event-plane resolution

The true Ψ is not directly observable. You reconstruct Ψ_{obs} from part of the event and correct v_n with the 'event-plane resolution' R = ■cos[n(Ψ_{obs} – Ψ_{true})]■, typically estimated from two sub-events. Without this correction v_n is underestimated by a factor of R (often 0.7–0.9).

6. Centrality binning

Define centrality from a reference observable (usually minimum-bias Σ E_T or total charged multiplicity):

```
// Step 1: fill an MB histogram of some centrality proxy X
// Step 2: find percentile cut points
double cuts[11];
for (int i=0;i<=10;++i) cuts[i] = hist->GetQuantiles( (10-i)/10. );
// Now cuts[0] = 100% (min), cuts[10] = 0% (max)
// bin 0 = most central, bin 9 = most peripheral
```

7. Typical dN_ch/dη pattern

Centrality %	dN_ch/dη |_{|η|<0.5} (Au+Au @ 200 GeV)
0–5%	~680
5–10%	~570
10–20%	~430
20–30%	~290
30–40%	~190
40–50%	~120
50–60%	~70
60–70%	~40
70–80%	~20

These are PHENIX values — HIJING with default settings reproduces them to within ~10%.

8. Common pitfalls

Pitfall	Symptom

Forgot <code>-fno-automatic</code>	Random crashes, non-reproducible events
Running min-bias but selecting tight b	Very few events survive centrality cut
Using particle-by-particle weights	Event-plane angle becomes biased
Not correcting for EP resolution	v_2 underestimated by 10–30%

9. Recommended output format for HIJING runs

Save: (a) event index, (b) impact parameter b, (c) N_part, N_coll, (d) particle arrays with PDG ID, (px, py, pz, E), production coordinates if you need them. Everything else can be derived.